

UNDERACTUATED ROBOTICS

Algorithms for Walking, Running, Swimming, Flying, and Manipulation

Russ Tedrake

© Russ Tedrake, 2024

Last modified 2024-12-6.

[How to cite these notes, use annotations, and give feedback.](#)

Note: These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube](#).

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

CHAPTER 21

Imitation Learning

Imitation learning, also known as "learning from demonstrations" (LfD), is the problem of learning a policy from a collection of demonstrations. For state-based feedback, these demonstrations take the form of a set of state-action sequences, $[\mathbf{x}[\cdot], \mathbf{u}[\cdot]]$. For the richer class of [output feedback](#), this takes the form of observation-action sequences, $[\mathbf{y}[\cdot], \mathbf{u}[\cdot]]$. Note that we do not require any explicit definition of a cost or reward function; in most cases we assume that the demonstrations are obtained from an optimal or near-optimal policy.

Broadly speaking, most approaches to imitation learning can be categorized as either *behavior cloning* (BC) or *inverse reinforcement learning* (IRL). Behavior cloning attempts to learn a policy directly from the data using supervised learning. Inverse RL (aka inverse optimal control) attempts to learn a cost function from the data, and then uses potentially more traditional optimal control approaches to synthesize a policy for this cost function, in the hopes of generalizing significantly beyond the demonstration data. See [[1](#), [2](#), [3](#)] for some fairly recent surveys, though admittedly they appeared a bit before the latest boom.

I think it's fair to say that today, in 2024, behavior cloning is once again taking the robotics world by storm (especially in manipulation research), and it now seems like the shortest path to building "generalist" robot foundation-model-style policies. We'll devote most of this chapter to it.

21.1 BEHAVIOR CLONING

Behavior cloning [4, 5] attempts to learn a policy directly from the policy's input-output data using supervised learning. In the case of full-state feedback, where we know that the optimal policy can be represented as a simple function of the state, this is typically cast as a simple regression problem, e.g. $\min_{\theta} \sum_i |\mathbf{u}_i - \pi_{\theta}(\mathbf{x}_i)|^2$, where θ are the parameters of e.g. the neural network. In the richer case of [output feedback](#), then this becomes a sequence learning problem (we are learning a dynamical system which transforms a sequence of observations into a sequence of actions).

Famously, Large Language Models (LLMs) are trained with behavior cloning (and then fine-tuned to make them more aligned with human preferences [6, 7]). OpenAI's GPT models are [autoregressive models](#) that predict the next token given a "context" of recent tokens [8, 9, 10, 11]. More recently, we've seen the extension to multi-modal models, such as visually-conditioned language models (VLMs) like Flamingo, GPT-4V, and LLaVa [12] with an increasing wealth of open-source reproductions [13, 14].

Is predicting actions fundamentally different from next-token prediction in language? There are a few reasons why it might be. Actions are continuous and high-dimensional, whereas language tokens are discrete. Our control systems get put into the feedback loop with physics, and have to deal with stochasticity from the environment that LLMs don't experience. Google DeepMind released the RT series of "vision-language-action" (VLA) models (RT-1 [15], RT-2 [16], and RT-X [17]) which started to show that predicting actions may not be so fundamentally different than language, at least for simple pick-and-place tasks. Then in 2023, there were two main lines of work which convincingly demonstrated this capability could even extend to surprisingly dexterous manipulation. Those were the [Diffusion Policy](#) [18] and the Action Chunking Transformer (ACT) from the [ALOHA project](#) [19]. Of course, these were built on a long line of progress on "visuomotor policies" which we'll discuss below.



前往平台观看：

Figure 21.1 - Diffusion Policy started as an intern project(!) by Cheng Chi at TRI, and blossomed into a great collaboration with Shuran Song.

Diffusion Policy and ALOHA seem to have been the watershed results for dexterous manipulation in robotics. Since that time, the internet is now rich with videos of highly dexterous manipulation from all sorts of robots ranging from very low cost manipulators up to and including humanoid robots with dexterous hands.

In one sense, it might seem a little disappointing that, after we've spent so much time in these notes exploring the rich mathematical foundations of dynamics and control, that behavior cloning from human teleop demonstrations can outperform our best methods for some class of problems (which are arguably more about understanding the world than about dynamics and control). But think of it this way: using supervised learning is an awfully clever way to explore the space of policy parameterizations, and has accelerated us into bigger questions about using cameras in the feedback loop, learning multitask/foundation models, and leveraging structure (such as 3D geometry / objectness) in our representations or not. The success of LLMs (and now multimodal models) is undeniable, and it would be a mistake to ignore the great new possibilities that have opened up. And I do think that there is something fundamentally about imitation learning for manipulation: in many cases the "rules of the game" (e.g. task specification) is not fully described by physics -- I've come to appreciate that humans bring an amazing amount of background knowledge / common sense to bear when they are performing even relatively simple manipulation tasks. I'm very confident that our knowledge of dynamics and control will help us penetrate the new vistas enabled by these high-capacity models.

Let me make one more important point. Sometimes people say that BC is fundamentally limited because it can never outperform the human demonstrator. But one of the early results from behavior cloning was that this limit is not strictly true. Certainly Chat-GPT can produce text output that by some metrics surpasses the capabilities of any one human. [Here](#) is a recent case study of this phenomenon for chess.

There is also a precedent for combining BC with other methods to improve beyond the original demonstrations. For DeepMind's AlphaGo[20], behavior cloning was only the first step -- it was enough to get us to a strong Go player, but not enough to be championship level. Adding "self-play" and improvement via Monte-Carlo tree search was fundamental to "mastering the game of Go".

21.1.1 Visuomotor policies (aka control from pixels)

In 2016, a few years after the start of the deep learning revolution, [21] introduced the concept of a "visuomotor policy", and produced [fairly stunning videos](#) of robots performing complicated tasks. The name "visuomotor" is chosen to emphasize that the policy is trained to predict actions directly from RGB camera images, using architectures similar to the one pictured below; in a sense this was a return to the early ideas of [4], but now powered with deep learning architectures and pretraining.

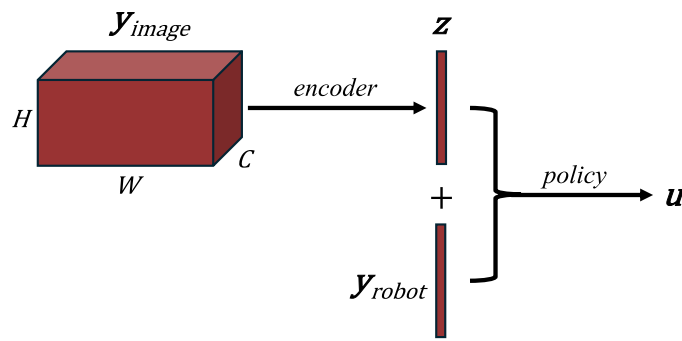


Figure 21.2 - The (now) classical visuomotor policy architecture, adapted from [21, 22]. The emphasis here is that z is a *learned state representation* which encodes the state of the environment.

My own journey with imitation learning started with a [project led by Pete Florence and Lucas Manuelli](#) on using a particular form of self-supervised learning for 3D geometries to train the policies [22]. For me, this was the first time I got to really experience directly how the model was trained, and I got to experience the incredibly impressive robustness that was possible in the roll-outs, even with a relatively modest amount of robot training data.

Writing feedback controllers which operate directly on the RGB camera images was something entirely new for me. We had been using RGB-D cameras to do some amount of visual state estimation / pose estimation before that, but were leaning pretty heavily on the depth channel and very explicit 3D reasoning/matching. But I remember around the time of our first imitation learning project, I asked the students "if you could only choose one, RGB *or* depth, which would you choose". They chose RGB. There are many situations where a task is unclear or even ambiguous when looking only at a depth image. The ambiguities can often be resolved with the addition of RGB, and there are many depth cues in RGB that allow us and our visuomotor policies to be successful for 3D tasks even without an explicit depth sensor. Diffusion Policy and ACT, for instance, achieve their amazing performance by consuming RGB (no depth).

In my experience, control theorists have very satisfying answers to almost any dynamics and control problem. But (with a few rare exceptions) they didn't do computer vision. This was something entirely new. The sensor model -- the mapping from, e.g. the state and parameters of a *MultibodyPlant* to the output image y -- is potentially a full game-engine-quality renderer. Even though there are lots of projects now on making differentiable renderers, these can only do so much because the pixelation process is inherently very local/non-smooth. Going from the image back into a manageable intermediate (latent) state representation, z , started becoming viable with the rise of deep networks for perception. Data/learning *does* feel fundamental here -- mapping from RGB into a meaningful representation for control is more about the statistics of natural scenes than about the model-based physics of propagating light.

This, for me, was the first main lesson I took from our imitation learning work: closing feedback loops by directly consuming RGB at control rates is now possible, and is incredibly powerful for robust performance in visually complex settings. Imitation learning (and reinforcement learning) have so far enabled this in a way that model-based control pipelines which require explicit state (or belief) estimation do not. (Of course, techniques like teacher-student distillation [23, 24] and [25] might help us bridge that gap.)

21.1.2 Behavior cloning as sequence modeling

21.1.3 Supervised learning in a feedback loop: dealing with distribution shift

One of the famous challenges in imitation learning is the problem of distribution shift. Imagine that you are training a policy for driving a car...

DAGGER [26]

Teacher/Student.

21.1.4 Dealing with suboptimal and multimodal demonstrations

21.2 ARCHITECTURES FOR VISUOMOTOR POLICIES

21.2.1 Desiderata

scalable, reliable training, ..., can cope with multimodality, ...

21.2.2 Output/action decoders

The visuomotor policies that we'll study here should output low-level robot actions -- **u** in the parlance of these notes. These need not be torque commands directly... in fact it's more typical for them to output a slightly higher-level command like joint velocity or end-effector velocity, which gets passed to a low-level controller. Note that there is also a now large body of literature where people use LLMs or VLMs to determine a sequence of high level actions, but assume that someone has authored or otherwise obtained a set of "skill libraries" that map the discrete high-level actions to control; while interesting, I would not call those approached visuomotor policies and will not discuss those here.

In all cases, the input encoders (discussed next) map the recent history of observations into some latent representation, which then eventually gets mapped back into actions via the action decoder. It is quite useful to categorize the different visuomotor policy architectures based on the different choices that they make about the action decoder.

There is a series of work, now commonly referred to as VLA (vision-language-action) architectures, which leverage the successful transformer architectures from language and vision by discretizing and tokenize the robot action space. Early examples include Decision Transformer [27], Gato[28], and more recently these powered the Robot

Transformer (RT) line of models from Google DeepMind: RT-1 [15], RT-2 [16], RT-X [17]. For example, in RT, the action is represented using a uniform discretization with 256 bins over each coordinate (using the 1st and 99th quantile of the actions in the training data as the minimum and maximum values, respectively); in those models the action space was taken to be the end-effector pose and gripper extension. Then they literally output a string of integers corresponding to those coordinates as desired text output for the VLM; if the integers not already represented as tokens in the VLM, then they simply overwrite the 256 least frequently used tokens [16]. You can find an strong open-source reproduction of RT-2X models in the [OpenVLA project](#) [29].

These tokenized-action architectures naturally learn a probability over next tokens, which deals very directly with the potential multimodality in the training data. But this comes at the cost of having discretized the action space. I don't worry much about the resolution of this discretization being limiting, but I'm a bit worried that the discretization destroys the natural inductive bias of the continuous space. (For instance, the end-effector position 0.1 is closer to 0.2 than to 0.5, but this information is completely discarded in the discretization.) Other people don't seem as concerned, and perhaps when we eventually have enough data it will all be in the noise.

Behavior Transformers (BeT) [30] and VQ-BeT [31] put more work into discretizing the action space, by preprocessing the data with k-means (in BeT) or Residual Vector Quantization (in VQ-BeT), and then doing some continuous up-sampling.

There were a number of attempts to handle multimodality in a more natively continuous setting. Implicit BC [32] called this out very explicitly, and also explored the use of Mixture Density Networks [33] for behavior cloning. But things really started working well when some of the new tools from generative AI started getting applied to robotics; most notably in the Action-chunking transformers (ACT) [19] which used Conditional [Variational Autoencoders](#) (CVAE) and in Diffusion Policy [18] which used Diffusion models. We'll discuss these in more detail below.

Both the ACT paper and the Diffusion Policy paper strongly emphasized another detail about the output encoding: rather than predicting a single (current) action to take, these models predicted an entire sequence of future actions, and then operate in a fashion similar to model-predictive control.

21.2.3 (Multi-modal) input encoders

Although researchers are rapidly adopting additional input modalities, by far the most common input modalities are the robot proprioception (e.g. joint sensors), which can be passed into the model directly, and image observations which need to be encoded from raw RGB into some intermediate representation. Although there is a torrent of literature on this, there are a few choices that have clearly emerged as the standards: ResNet and ViT. For instance, the original Diffusion Policy paper used a ResNet-18 (without pretraining) with small modifications, e.g. to maintain spatial information

[18], but as the scale of the experiments has increased we have seen more success with CLIP-pretrained ViT [34].

Language-conditioned multitask policies... [17, 29].

21.3 DIFFUSION POLICY

One particularly successful form of behavior cloning for visuomotor policies with continuous action spaces is the [Diffusion Policy](#) [18]. The dexterous manipulation team at TRI had been working on behavior cloning for some time, but the Diffusion Policy (which started as a summer internship project!) architecture has allowed us to very reliably train [incredibly dexterous tasks](#) and really start to scale up our ambitions for manipulation.

21.3.1 Denoising Diffusion models

"Denoising Diffusion" models are an approach to generative AI, made famous by their ability to generate high-resolution photorealistic images. Inspired by the "manifold hypothesis" (e.g. the idea that realistic images live on a low-dimensional manifold in pixel space), the intuition behind denoising diffusion is that we train a model by adding noise to samples drawn from the data distribution, then learn to predict the noise from the noisy images, in order to "denoise" random images back on to the manifold. While image generation made these models famous, they have proven to be highly capable in generating samples from a wide variety of high-dimension continuous distributions, even distributions that are conditioned on high-dimensional inputs. I recommend [this blog post](#) and [35] as excellent introductions.

Let's consider samples $\mathbf{u} \in \mathbb{R}^m$ drawn from a training dataset $\mathcal{D}$. Diffusion models are trained to estimate a noise vector $\epsilon \in \mathbb{R}^m$ to minimize the loss function

$$\ell(\theta) = \mathbb{E}_{\mathbf{u}, \epsilon, \sigma} \|\mathbf{f}_\theta(\mathbf{u} + \sigma\epsilon, \sigma) - \epsilon\|^2,$$

where θ is the parameter vector, and $\mathbf{f}_\theta$ is typically some high-capacity neural network. In practice, training is done by randomly sampling $\mathbf{u}$ from $\mathcal{D}$, ϵ from $\mathcal{N}(\mathbf{0}_m, \mathbf{I}_{m \times m})$, and σ from a uniform distribution over a positive set of numbers denoted as $\{\sigma_k\}_{k=0}^K$, where we have $\sigma_k > \sigma_{k-1}$.

To sample a new output from the model, the denoising diffusion implicit models (DDIM) sampler [36] takes multiple steps:

$$\mathbf{u}_{k-1} = \mathbf{u}_k + (\sigma_{k-1} - \sigma_k) \mathbf{f}_\theta(\mathbf{u}_k, \sigma_k).$$

This specific parameterization of the update (and my preferred notation more generally) comes from [37]. I've presented the deterministic denoiser here, but some analysis suggests that Langevin sampling can improve performance if one takes many denoising steps[38].

Diffusion models have a slightly convoluted history. The term "diffusion" came from a paper [39] which used an analogy from thermodynamics to use a prescribed diffusion process to slowly transform data into random noise, and then learned to reverse this procedure by training an inverse diffusion. Well before that, a series of work starting with [40] studied the problem of learning the score function (the gradient of the log probability of a distribution) of a data distribution, and [41] made a connection to denoising autoencoders. [42] put all of this together beautifully and combined it with deep learning to propose denoising diffusion as a generative modeling technique. They learned a single network that was conditioned on the noise level. This was followed quickly by [43] which introduced denoise diffusion probabilistic models (DDPM) using an even simpler update and showed results competitive with other leading generative modeling techniques leading to [36] giving us the DDIM update above. [37] gives a deterministic interpretation as learning the distance function from the data manifold, and sampling as performing approximate gradient descent on this function.

It is straight-forward to condition the generative model on an exogeneous input, by simply adding an additional signal, $\mathbf{y}$, to the denoiser: $f_{\theta}(\mathbf{u}, \sigma, \mathbf{y})$.

21.3.2 Diffusion Policy

Behavior cloning is perhaps the simplest form of imitation learning -- it simply attempts learn a policy using supervised learning to match expert demonstrations. While it is tempting to learn deterministic output-feedback policies (maps from history of observations to actions), one quickly finds that human demonstrations are typically not unique. Perhaps this is not surprising, as we know that optimal feedback policies in general are not unique! To address this non-uniqueness / multi-modality in the human demonstrations, it's well understood that behavior cloning benefits from learning a conditional *distribution* over actions.

Diffusion Policy is the natural application of (conditional) denoising diffusion models to learning these policies. It was inspired, in particular, but the modeling choices in Diffuser[44]. In particular, rather than generating a single action, the denoiser in diffusion policy outputs a sequence of actions with horizon H_u ; like [19] we found experimentally that this leads to more stable roll-outs. [45] provides some possible theoretical justification for this choice. We condition the input on a history of observations of length H_y .

21.3.3 Diffusion Policy for Linear Policies

Let me be clear, it almost certainly does *not* make sense to use a diffusion policy to represent a linear (output) feedback control policy. But it can be helpful to understand what the Diffusion Policy looks like in this extremely simplified case.

Consider the case where we have the standard linear-Gaussian dynamical system:

$$\begin{aligned}\mathbf{x}[n+1] &= \mathbf{A}\mathbf{x}[n] + \mathbf{B}\mathbf{u}[n] + \mathbf{w}[n], \\ \mathbf{y}[n] &= \mathbf{C}\mathbf{x}[n] + \mathbf{D}\mathbf{u}[n] + \mathbf{v}[n], \\ \mathbf{w}[n] &\sim \mathcal{N}(\mathbf{0}, \Sigma_w), \quad \mathbf{v}[n] \sim \mathcal{N}(\mathbf{0}, \Sigma_v).\end{aligned}$$

Imagine that we create a dataset by rolling out trajectory demonstrations using a linear (output) feedback policy -- it could be, for instance, the optimal policy from LQG design. The question is: what (exactly) does the diffusion policy learn?

State-feedback

Let's start with the state-feedback case, where we generate roll-outs using a controller, $\mathbf{u} = -\mathbf{K}\mathbf{x}$, given a Gaussian distribution of initial conditions and Gaussian process noise. In this case, the training loss function reduces to

$$\ell(\theta) = \mathbb{E}_{\mathbf{x}, \epsilon, \sigma} \|\mathbf{f}_\theta(-\mathbf{K}\mathbf{x} + \sigma\epsilon, \sigma, \mathbf{x}) - \epsilon\|^2,$$

where the expectation in $\mathbf{x}$ is over the stationary distribution of the closed-loop system. In this case, we don't need a neural network; take $\mathbf{f}_\theta$ to be a simple function. In particular, we can achieve zero loss using the denoiser given by

$$\mathbf{f}_\theta(\mathbf{u}, \sigma, \mathbf{x}) = \frac{1}{\sigma}[\mathbf{u} + \mathbf{K}\mathbf{x}].$$

At evaluation time, the sampling iterations,

$$\mathbf{u}_{k-1} = \mathbf{u}_k + \frac{\sigma_{k-1} - \sigma_k}{\sigma_k}[\mathbf{u}_k + \mathbf{K}\mathbf{x}],$$

will converge on $\mathbf{u}_0 = -\mathbf{K}\mathbf{x}$. (Clearly $\mathbf{u}_k = -\mathbf{K}\mathbf{x}$ is a fixed point of the iteration, and the $\frac{\sigma_{k-1} - \sigma_k}{\sigma_k}$ term is like the step-size of gradient descent.)

Output-feedback

Returning to output feedback, we know that the optimal linear output-feedback policy from LQG is typically written in a state-space form, e.g.:

$$\begin{aligned}\hat{\mathbf{x}}[n+1] &= \mathbf{A}\hat{\mathbf{x}}[n] + \mathbf{B}_c\mathbf{u}[n] + \mathbf{L}(\mathbf{y}[n] - \mathbf{C}\hat{\mathbf{x}}[n] - \mathbf{D}\mathbf{u}[n]), \\ \mathbf{u}[n] &= -\mathbf{K}\hat{\mathbf{x}}[n].\end{aligned}$$

But the Diffusion Policy architecture (with $H_u = 1$) is typically formulated as learning a denoiser conditioned on a finite history of actions and observations,

$$\begin{aligned}f_\theta(\mathbf{u}[n], \sigma, \bar{\mathbf{y}}_{H_y}, \bar{\mathbf{u}}_{H_y}), \\ \bar{\mathbf{y}}_{H_y} &= [\mathbf{y}[n-1], \dots, \mathbf{y}[n-H_y]], \\ \bar{\mathbf{u}}_{H_y} &= [\mathbf{u}[n-1], \dots, \mathbf{u}[n-H_y]].\end{aligned}$$

So the more direct analogy would be to generating demonstrations from an autoregressive linear policy, e.g.

$$\mathbf{u}[n] = -\mathbf{K} \begin{bmatrix} \bar{\mathbf{y}}_{H_y} \\ \bar{\mathbf{u}}_{H_y} \end{bmatrix}.$$

This autoregressive form is commonly used in [disturbance-based](#) output feedback, and may be helpful to think of it e.g. as an "unrolled" (truncated) Kalman filter. Again, we can achieve zero loss with a denoiser of the form

$$\mathbf{f}_\theta(\mathbf{u}, \sigma, \bar{\mathbf{y}}_{H_y}, \bar{\mathbf{u}}_{H_y}) = \frac{1}{\sigma} \left[\mathbf{u} + \mathbf{K} \begin{bmatrix} \bar{\mathbf{y}}_{H_y} \\ \bar{\mathbf{u}}_{H_y} \end{bmatrix} \right].$$

This shapes my current mental model for what Diffusion Policy is learning, even in complicated manipulation settings: it may be helpful to think about the history of actions and observations being compressed into a (task-relevant) belief-state representation, like we would have in an (unrolled, truncated) Kalman filter, that is sufficient for predicting actions.

Action sequence prediction

Another important aspect of the Diffusion Policy architecture is that it is predicting not just the instantaneous action, but a sequence of future actions. Even in the linear policy setting, we can see that this puts different pressure on the learned representations...

21.4 [INVERSE REINFORCEMENT LEARNING](#)

21.5 [VISTAS](#)

21.5.1 Multitask / foundation models for control

If control directly from pixels was the first capability unlocked by imitation learning, I would say that large-scale multitask decision making is the second. Multitask learning has a long history in the machine learning community [\[46\]](#), and multitask learning for control was popular first in "multitask RL". But the rise of foundation models [\[47\]](#) in language and now in multimodal models, has completely changed our capabilities and our expectations. Suddenly, one can imagine a single policy, represented in a high-capacity neural network model, whose inputs are the robot sensors (including RGB cameras) plus a natural language command ("Robot, please make me a pizza"), and for the first time ever it seems like something like this could plausibly work.

In my view, this has potentially profound implications for how we think about control. Our basic control definitions start with, e.g. we have a state $\mathbf{x}$, inputs $\mathbf{u}$, outputs $\mathbf{y}$. The discussion on output feedback got us thinking a little about state representations for control -- for instance a belief state is a sufficient (but not necessary) state because it is a sufficient statistic of the history of actions and observations. But multitask in the

imitation learning setting changes things. In the simple case, we'll say that our inputs $\mathbf{u}$, and outputs $\mathbf{y}$ are the same across all of the tasks. But it may well be that the underlying state space is not. (I admit that philosophically there is a state of the entire universe which is the same across tasks, but I mean the more tractable representations of state that we've been using through the notes.) What does it mean to learn a state representation for control across tasks where even the dimension/cardinality of the state space can be different? Even our catch-all definition of belief state breaks down in this case.

Are there tractable ways to describe distributions over tasks that are amenable to our strongest theoretical tools, but still relevant for the complexity and diversity of the real world? When we talked about stochastic optimal control, we gave examples where taking an average over many possible rollouts can actually simplify the loss landscape, avoiding some local minima and making optimization easier. Can multitask control formulations have a similar effect?

Going further, how exactly is it that solving/learning one task can potentially help us in solving/learning another? This brings up basic questions about designing a curriculum for our control systems. Is it possible for us to soar to higher and higher heights if we sequence our control problem instances correctly?

21.5.2 Distributed decentralized learning (aka "fleet learning")

When I start using phrases like "learning" and "curriculum", then it becomes very natural to think in terms of our natural intelligence. How did we learn to walk? To play tennis? But let's remember that these analogies only go so far. For me, the GPT series of models are clearly unlike any single natural intelligence, they are more like a collective intelligence of the entire species (though still certainly deficient in some metrics). In the age of foundation models, it may not be the case that every robot needs to learn to use a toaster; the dream of "fleet learning" is that one robot will learn how to use a toaster and then they will all have learned.

This brings up fundamental questions about the learning algorithms, about data efficiency (and privacy). But it also challenges our theories of dynamics and control. For instance, there are open questions about how to balance being a generalist and using only shared data vs being a specialist. Certainly if a particular robot is solving problems in a particular warehouse, then while the statistics of tasks across the world may help form robust representations, this robot can almost certainly perform better if it narrows and specializes the policies (and world models) to exploit the distribution of tasks in the warehouse.

A particular version of this question appears in the context of "cross-embodiment" data and models. Right now, robot data with action labels is scarce (compared with online data for text and images). This, in part, has motivated the use of datasets which combine data from many robots/platforms [48, 49], and architectures that can share

representations and transfer learning even when the number/types of sensors and actuators changes across the fleet.

21.5.3 Be rigorous

The fact that many of these fundamental questions are now being asked makes this a simply amazing time to be a roboticist. However, the pace of new innovations is so fast that often researchers feel pressure to race to publication before having done proper rigorous theoretical or empirical work. We are building tall towers but with somewhat shakey foundations. I firmly believe that the tenants of dynamics and control (amongst other rigorous technical tools) have a lot to contribute to understanding and continuing to push the field forward, and that some of the maturity with which we can understand these simpler (but not very simple!) problems can serve as a model for what we should expect about our understanding of the even more complex ones.

REFERENCES

1. Brenna D Argall and Sonia Chernova and Manuela Veloso and Brett Browning, "A survey of robot learning from demonstration", *Robotics and autonomous systems*, vol. 57, no. 5, pp. 469--483, 2009.
2. J.A. Bagnell, "An Invitation to Imitation", Tech. Report, CMU-RI-TR-15-08, March, 2015.
3. Takayuki Osa and Joni Pajarinen and Gerhard Neumann and J Andrew Bagnell and Pieter Abbeel and Jan Peters and others, "An algorithmic perspective on imitation learning", *Foundations and Trends in Robotics*, vol. 7, no. 1-2, pp. 1-179, 2018.
4. Dean A Pomerleau, "Alvinn: An autonomous land vehicle in a neural network", *Advances in neural information processing systems*, vol. 1, 1988.
5. Michael Bain and Claude Sammut, "A Framework for Behavioural Cloning.", *Machine Intelligence 15*, pp. 103--129, 1995.
6. Daniel M Ziegler and Nisan Stiennon and Jeffrey Wu and Tom B Brown and Alec Radford and Dario Amodei and Paul Christiano and Geoffrey Irving, "Fine-tuning language models from human preferences", *arXiv preprint arXiv:1909.08593*, 2019.
7. Shengyu Zhang and Linfeng Dong and Xiaoya Li and Sen Zhang and Xiaofei Sun and Shuhe Wang and Jiwei Li and Runyi Hu and Tianwei Zhang and Fei Wu and Guoyin Wang, "Instruction Tuning for Large Language Models: A Survey", , 2024.

8. Alec Radford and Karthik Narasimhan and Tim Salimans and Ilya Sutskever and others, "Improving language understanding by generative pre-training", , 2018.
9. Alec Radford and Jeffrey Wu and Rewon Child and David Luan and Dario Amodei and Ilya Sutskever and others, "Language models are unsupervised multitask learners", *OpenAI blog*, vol. 1, no. 8, pp. 9, 2019.
10. Tom Brown and Benjamin Mann and Nick Ryder and Melanie Subbiah and Jared D Kaplan and Prafulla Dhariwal and Arvind Neelakantan and Pranav Shyam and Girish Sastry and Amanda Askell and others, "Language models are few-shot learners", *Advances in neural information processing systems*, vol. 33, pp. 1877-1901, 2020.
11. Sébastien Bubeck and Varun Chandrasekaran and Ronen Eldan and Johannes Gehrke and Eric Horvitz and Ece Kamar and Peter Lee and Yin Tat Lee and Yuanzhi Li and Scott Lundberg and others, "Sparks of artificial general intelligence: Early experiments with gpt-4", *arXiv preprint arXiv:2303.12712*, 2023.
12. Haotian Liu and Chunyuan Li and Qingyang Wu and Yong Jae Lee, "Visual Instruction Tuning", , 2023.
13. Siddharth Karamcheti and Suraj Nair and Ashwin Balakrishna and Percy Liang and Thomas Kollar and Dorsa Sadigh, "Prismatic vlms: Investigating the design space of visually-conditioned language models", *arXiv preprint arXiv:2402.07865*, 2024.
14. Hugo Laurençon and Léo Tronchon and Matthieu Cord and Victor Sanh, "What matters when building vision-language models?", , 2024.
15. Anthony Brohan and Noah Brown and Justice Carbajal and Yevgen Chebotar and Joseph Dabis and Chelsea Finn and Keerthana Gopalakrishnan and Karol Hausman and Alex Herzog and Jasmine Hsu and Julian Ibarz and Brian Ichter and Alex Irpan and Tomas Jackson and Sally Jesmonth and Nikhil Joshi and Ryan Julian and Dmitry Kalashnikov and Yuheng Kuang and Isabel Leal and Kuang-Huei Lee and Sergey Levine and Yao Lu and Utsav Malla and Deeksha Manjunath and Igor Mordatch and Ofir Nachum and Carolina Parada and Jodilyn Peralta and Emily Perez and Karl Pertsch and Jornell Quiambao and Kanishka Rao and Michael Ryoo and Grecia Salazar and Pannag Sanketi and Kevin Sayed and Jaspiar Singh and Sumedh Sontakke and Austin Stone and Clayton Tan and Huong Tran and Vincent Vanhoucke and Steve Vega and Quan Vuong and Fei Xia and Ted Xiao and Peng Xu and Sichun Xu and Tianhe Yu and Brianna Zitkovich,

"RT-1: Robotics Transformer for Real-World Control at Scale", *arXiv preprint arXiv:2212.06817*, 2022.

16. Anthony Brohan and Noah Brown and Justice Carbajal and Yevgen Chebotar and Xi Chen and Krzysztof Choromanski and Tianli Ding and Danny Driess and Avinava Dubey and Chelsea Finn and Pete Florence and Chuyuan Fu and Montse Gonzalez Arenas and Keerthana Gopalakrishnan and Kehang Han and Karol Hausman and Alex Herzog and Jasmine Hsu and Brian Ichter and Alex Irpan and Nikhil Joshi and Ryan Julian and Dmitry Kalashnikov and Yuheng Kuang and Isabel Leal and Lisa Lee and Tsang-Wei Edward Lee and Sergey Levine and Yao Lu and Henryk Michalewski and Igor Mordatch and Karl Pertsch and Kanishka Rao and Krista Reymann and Michael Ryoo and Grecia Salazar and Pannag Sanketi and Pierre Sermanet and Jaspier Singh and Anikait Singh and Radu Soricut and Huong Tran and Vincent Vanhoucke and Quan Vuong and Ayzaan Wahid and Stefan Welker and Paul Wohlhart and Jialin Wu and Fei Xia and Ted Xiao and Peng Xu and Sichun Xu and Tianhe Yu and Brianna Zitkovich, "RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control", *arXiv preprint arXiv:2307.15818*, 2023.
17. Abby O 欒 檣 eill and Abdul Rehman and Abhiram Maddukuri and Abhishek Gupta and Abhishek Padalkar and Abraham Lee and Acorn Pooley and Agrim Gupta and Ajay Mandlekar and Ajinkya Jain and others, "Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0", 2024 *IEEE International Conference on Robotics and Automation (ICRA)* , pp. 6892-6903, 2024.
18. Cheng Chi and Zhenjia Xu and Siyuan Feng and Eric Cousineau and Yilun Du and Benjamin Burchfiel and Russ Tedrake and Shuran Song, "Diffusion Policy: Visuomotor Policy Learning via Action Diffusion", , 2024.
19. Tony Z Zhao and Vikash Kumar and Sergey Levine and Chelsea Finn, "Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware", *arXiv preprint arXiv:2304.13705*, 2023.
20. David Silver and Aja Huang and Chris J Maddison and Arthur Guez and Laurent Sifre and George Van Den Driessche and Julian Schrittwieser and Ioannis Antonoglou and Veda Panneershelvam and Marc Lanctot and others, "Mastering the game of Go with deep neural networks and tree search", *nature*, vol. 529, no. 7587, pp. 484--489, 2016.
21. Sergey Levine and Chelsea Finn and Trevor Darrell and Pieter Abbeel, "End-to-end training of deep visuomotor policies", *The Journal of Machine Learning Research*, vol. 17, no. 1, pp. 1334--1373, 2016.

22. Peter Florence and Lucas Manuelli and Russ Tedrake, "Self-Supervised Correspondence in Visuomotor Policy Learning", *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 492-499, April, 2020. [[link](#)]
23. Dian Chen and Brady Zhou and Vladlen Koltun and Philipp Krahenbuhl, "Learning by cheating", *Conference on Robot Learning*, pp. 66--75, 2020.
24. Jemin Hwangbo and Joonho Lee and Alexey Dosovitskiy and Dario Bellicoso and Vassilios Tsounis and Vladlen Koltun and Marco Hutter, "Learning agile and dynamic motor skills for legged robots", *Science Robotics*, vol. 4, no. 26, pp. eaau5872, 2019.
25. Glen Chou and Russ Tedrake, "Synthesizing Stable Reduced-Order Visuomotor Policies for Nonlinear Systems via Sums-of-Squares Optimization", *Proceedings of the IEEE 62nd Annual Conference on Decision and Control (CDC)*, 2023. [[link](#)]
26. Stéphane Ross and Geoffrey Gordon and Drew Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning", *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pp. 627--635, 2011.
27. Lili Chen and Kevin Lu and Aravind Rajeswaran and Kimin Lee and Aditya Grover and Misha Laskin and Pieter Abbeel and Aravind Srinivas and Igor Mordatch, "Decision transformer: Reinforcement learning via sequence modeling", *Advances in neural information processing systems*, vol. 34, pp. 15084--15097, 2021.
28. Scott Reed and Konrad Zolna and Emilio Parisotto and Sergio Gomez Colmenarejo and Alexander Novikov and Gabriel Barth-Maron and Mai Gimenez and Yury Sulsky and Jackie Kay and Jost Tobias Springenberg and others, "A generalist agent", *arXiv preprint arXiv:2205.06175*, 2022.
29. Moo Jin Kim and Karl Pertsch and Siddharth Karamcheti and Ted Xiao and Ashwin Balakrishna and Suraj Nair and Rafael Rafailov and Ethan Foster and Grace Lam and Pannag Sanketi and Quan Vuong and Thomas Kollar and Benjamin Burchfiel and Russ Tedrake and Dorsa Sadigh and Sergey Levine and Percy Liang and Chelsea Finn, "OpenVLA: An Open-Source Vision-Language-Action Model", *arXiv preprint arXiv:2406.09246*, 2024. [[link](#)]
30. Nur Muhammad Shafiullah and Zichen Cui and Ariuntuya (Arty) Altanzaya and Lerrel Pinto, "Behavior Transformers: Cloning k modes with one stone",

Advances in Neural Information Processing Systems , vol. 35, pp. 22955--22968, 2022.

31. Seungjae Lee and Yibin Wang and Haritheja Etukuru and H Jin Kim and Nur Muhammad Mahi Shafiullah and Lerrel Pinto, "Behavior generation with latent actions", *arXiv preprint arXiv:2403.03181*, 2024.
32. Pete Florence and Corey Lynch and Andy Zeng and Oscar A Ramirez and Ayzaan Wahid and Laura Downs and Adrian Wong and Johnny Lee and Igor Mordatch and Jonathan Tompson, "Implicit behavioral cloning", *Conference on Robot Learning* , pp. 158--168, 2022.
33. Christopher M Bishop, "Mixture density networks", , 1994.
34. Cheng Chi and Zhenjia Xu and Chuer Pan and Eric Cousineau and Benjamin Burchfiel and Siyuan Feng and Russ Tedrake and Shuran Song, "Universal Manipulation Interface: In-The-Wild Robot Teaching Without In-The-Wild Robots", , 2024.
35. Preetum Nakkiran and Arwen Bradley and Hattie Zhou and Madhu Advani, "Step-by-Step Diffusion: An Elementary Tutorial", , 2024.
36. Jiaming Song and Chenlin Meng and Stefano Ermon, "Denoising Diffusion Implicit Models", *International Conference on Learning Representations* , 2020.
37. Frank Permenter and Chenyang Yuan, "Interpreting and Improving Diffusion Models Using the Euclidean Distance Function", *arXiv preprint arXiv:2306.04848*, 2023.
38. Yilun Xu and Mingyang Deng and Xiang Cheng and Yonglong Tian and Ziming Liu and Tommi Jaakkola, "Restart Sampling for Improving Generative Processes", *Advances in Neural Information Processing Systems* , vol. 36, pp. 76806--76838, 2023.
39. Jascha Sohl-Dickstein and Eric Weiss and Niru Maheswaranathan and Surya Ganguli, "Deep unsupervised learning using nonequilibrium thermodynamics", *International conference on machine learning* , pp. 2256--2265, 2015.
40. Aapo Hyvarinen, "Estimation of Non-Normalized Statistical Models by Score Matching", *Journal of Machine Learning Research*, vol. 6, pp. 695--708, 2005.
41. Pascal Vincent, "A connection between score matching and denoising autoencoders", *Neural computation*, vol. 23, no. 7, pp. 1661--1674, 2011.

42. Yang Song and Stefano Ermon, "Generative Modeling by Estimating Gradients of the Data Distribution", *Advances in Neural Information Processing Systems*, vol. 32, 2019.
43. Jonathan Ho and Ajay Jain and Pieter Abbeel, "Denoising diffusion probabilistic models", *Advances in neural information processing systems*, vol. 33, pp. 6840-6851, 2020.
44. Michael Janner and Yilun Du and Joshua Tenenbaum and Sergey Levine, "Planning with Diffusion for Flexible Behavior Synthesis", *International Conference on Machine Learning*, pp. 9902--9915, 2022.
45. Adam Block and Ali Jadbabaie and Daniel Pfrommer and Max Simchowitz and Russ Tedrake, "Provable Guarantees for Generative Behavior Cloning: Bridging Low-Level Stability and High-Level Behavior", *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
46. Yu Zhang and Qiang Yang, "A survey on multi-task learning", *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 12, pp. 5586--5609, 2021.
47. Rishi Bommasani and Drew A Hudson and Ehsan Adeli and Russ Altman and Simran Arora and Sydney von Arx and Michael S Bernstein and Jeannette Bohg and Antoine Bosselut and Emma Brunskill and others, "On the opportunities and risks of foundation models", *arXiv preprint arXiv:2108.07258*, 2021.
48. Dhruv Shah and Ajay Sridhar and Nitish Dashora and Kyle Stachowicz and Kevin Black and Noriaki Hirose and Sergey Levine, "ViNT: A Foundation Model for Visual Navigation", *Conference on Robot Learning*, pp. 711--733, 2023.
49. Abhishek Padalkar and Acorn Pooley and Ajinkya Jain and Alex Bewley and Alex Herzog and Alex Irpan and Alexander Khazatsky and Anant Rai and Anikait Singh and Anthony Brohan and others, "Open x-embodiment: Robotic learning datasets and rt-x models", *arXiv preprint arXiv:2310.08864*, 2023.